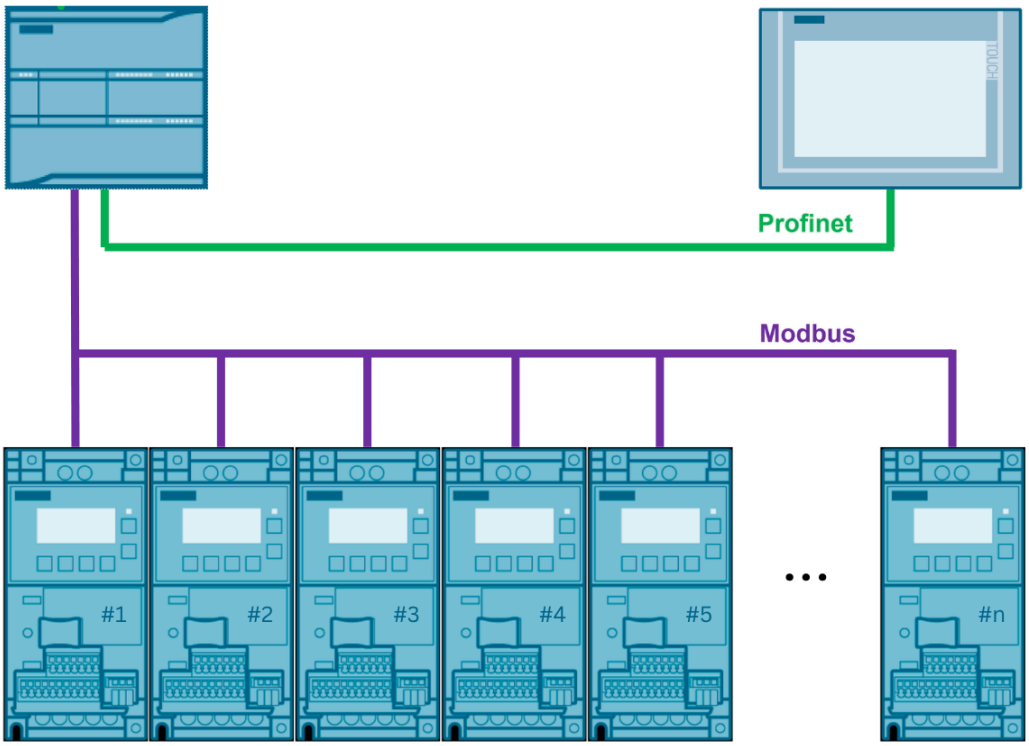
Runtime DS47 Parameter Management for SINAMICS V20 Using Siemens PLC and Modbus RTU



1. Task

The objective of this project is to implement a scalable DS47 acyclic communication framework between SIMATIC S7-1200 PLC and multiple SINAMICS V20 drives via Modbus RTU.

The framework supports runtime parameter reading, parameter writing, diagnostics, and HMI-based drive commissioning.

2. Hardware and Software Components

Hardware Components

- Siemens S7-1200 PLC (CPU 1214C AC/DC/RLY)
- CB 1241 RS485 communication board
- SINAMICS V20 Variable Frequency Drives
- SIMATIC HMI Basic Panel (KTP700 Basic PN)

Software Components

- Tia Portal V20 Upd3

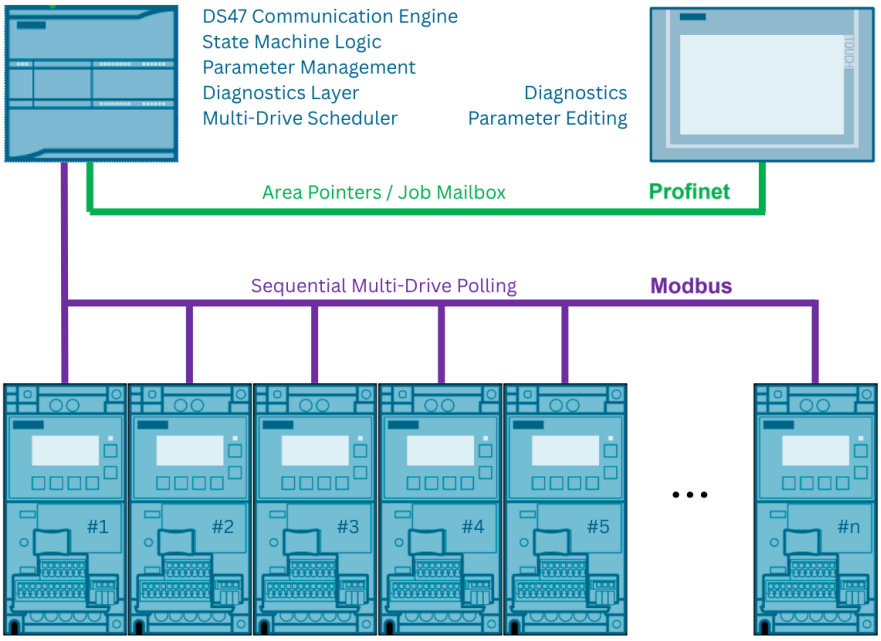
3. Solution Overview

The implemented solution provides a centralized parameter management framework for multiple SINAMICS V20 drives using a Siemens S7-1200 PLC and SIMATIC HMI.

The PLC acts as the main communication controller and handles both standard cyclic drive polling and DS47 acyclic parameter transactions over Modbus. The HMI is used as the operator interface for drive selection, runtime parameter monitoring, parameter editing, and diagnostics.

The communication architecture is based on a sequential processing model. Only one drive is processed at a time, which avoids transaction conflicts on the shared Modbus channel and keeps the communication behavior deterministic.

The framework supports runtime access to integer and floating-point drive parameters, structured request/response handling, response validation, and basic error interpretation. This allows drive parameters to be monitored and modified from the HMI without direct interaction with the local drive keypad or external commissioning software.



Acyclic Read Workflow:
Read Request Trigger
Build DS47 Read Request
Write Request to Drive
Response Polling
Response Validation
Parameter Data Conversion
Next Parameter / Complete

Acyclic Write Workflow:
Write Request Trigger
Build DS47 Write Request
Write Request to Drive
Response Polling
Write Response Validation
Error Interpretation
Complete

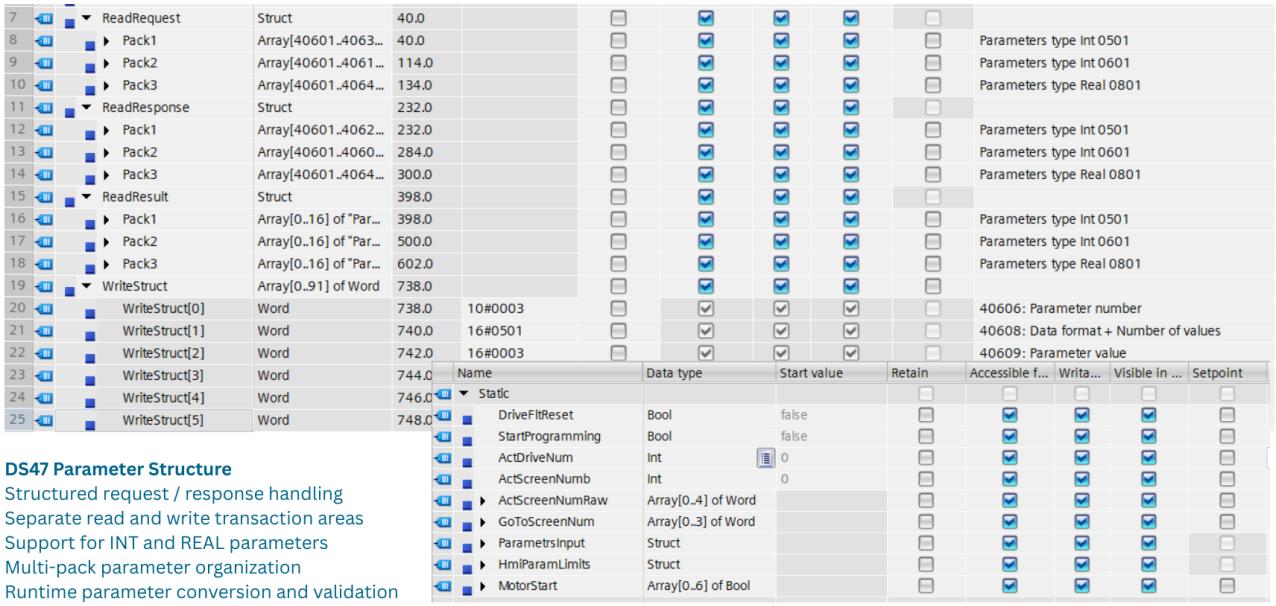
Supported Parameter Types:
Integer Parameters
Floating-Point Parameters
Multi-Pack Parameter Structures

Fault Handling:
Communication Recovery
DS47 Response Validation
Drive Error Interpretation
Parameter Error Handling

4. Communication Architecture

The communication architecture was designed to support runtime parameter access to multiple SINAMICS V20 drives using DS47 acyclic communication over Modbus RTU.

The solution separates cyclic drive polling from acyclic parameter transactions and implements dedicated request, response, and result structures for deterministic communication handling.



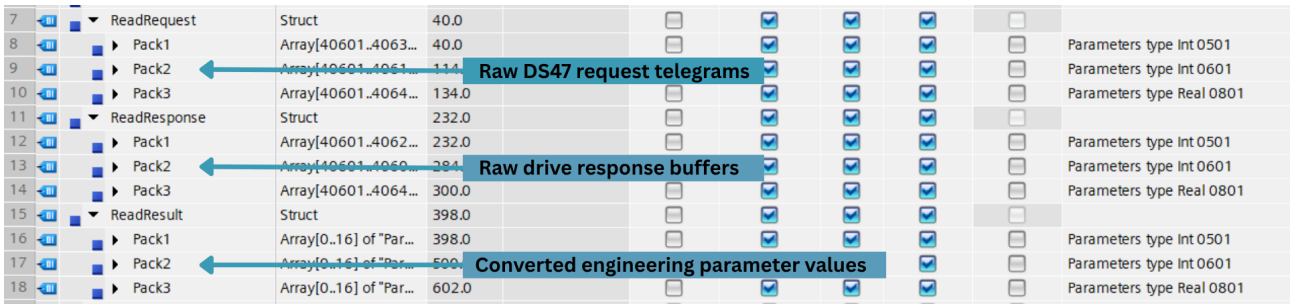
7	ReadRequest	Struct	40.0									
8	Pack1	Array[40601..4063...	40.0									Parameters type Int 0501
9	Pack2	Array[40601..4061...	114.0									Parameters type Int 0601
10	Pack3	Array[40601..4064...	134.0									Parameters type Real 0801
11	ReadResponse	Struct	232.0									
12	Pack1	Array[40601..4062...	232.0									Parameters type Int 0501
13	Pack2	Array[40601..4060...	284.0									Parameters type Int 0601
14	Pack3	Array[40601..4064...	300.0									Parameters type Real 0801
15	ReadResult	Struct	398.0									
16	Pack1	Array[0..16] of "Par...	398.0									Parameters type Int 0501
17	Pack2	Array[0..16] of "Par...	500.0									Parameters type Int 0601
18	Pack3	Array[0..16] of "Par...	602.0									Parameters type Real 0801
19	WriteStruct	Array[0..91] of Word	738.0									
20	WriteStruct[0]	Word	738.0	10#0003								40606: Parameter number
21	WriteStruct[1]	Word	740.0	16#0501								40608: Data format + Number of values
22	WriteStruct[2]	Word	742.0	16#0003								40609: Parameter value
23	WriteStruct[3]	Word	744.0									
24	WriteStruct[4]	Word	746.0									
25	WriteStruct[5]	Word	748.0									

Name	Data type	Start value	Retain	Accessible f...	Writa...	Visible in ...	Setpoint
Static							
DriveFltReset	Bool	false					
StartProgramming	Bool	false					
ActDriveNum	Int	0					
ActScreenNumb	Int	0					
ActScreenNumRaw	Array[0..4] of Word						
GoToScreenNum	Array[0..3] of Word						
ParametrsInput	Struct						
HmiParamLimits	Struct						
MotorStart	Array[0..6] of Bool						

DS47 Parameter Structure

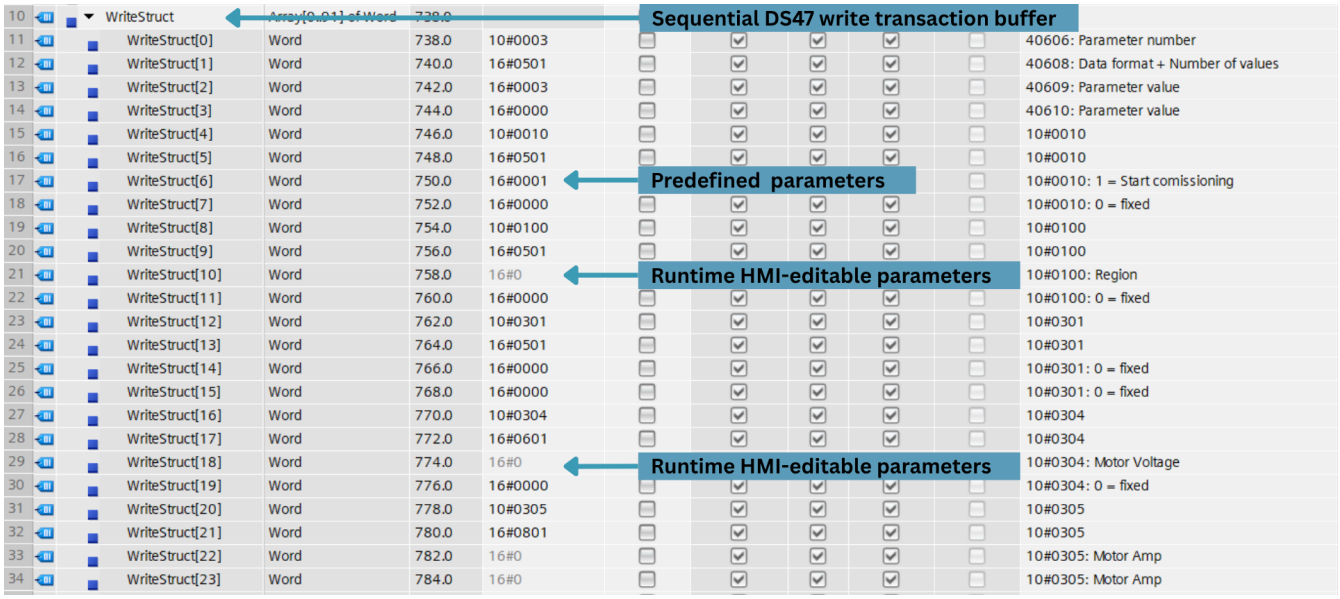
Structured request / response handling
Separate read and write transaction areas
Support for INT and REAL parameters
Multi-pack parameter organization
Runtime parameter conversion and validation

Read operations are grouped into parameter packs, while write operations are executed sequentially one parameter at a time to maintain deterministic communication handling.



7	ReadRequest	Struct	40.0		
8	Pack1	Array[40601..4063...	40.0		Parameters type Int 0501
9	Pack2	Array[40601..4061...	114.0	Raw DS47 request telegrams	Parameters type Int 0601
10	Pack3	Array[40601..4064...	134.0		Parameters type Real 0801
11	ReadResponse	Struct	232.0		
12	Pack1	Array[40601..4062...	232.0		Parameters type Int 0501
13	Pack2	Array[40601..4060...	284.0	Raw drive response buffers	Parameters type Int 0601
14	Pack3	Array[40601..4064...	300.0		Parameters type Real 0801
15	ReadResult	Struct	398.0		
16	Pack1	Array[0..16] of "Par...	398.0		Parameters type Int 0501
17	Pack2	Array[0..16] of "Par...	500.0	Converted engineering parameter values	Parameters type Int 0601
18	Pack3	Array[0..16] of "Par...	602.0		Parameters type Real 0801

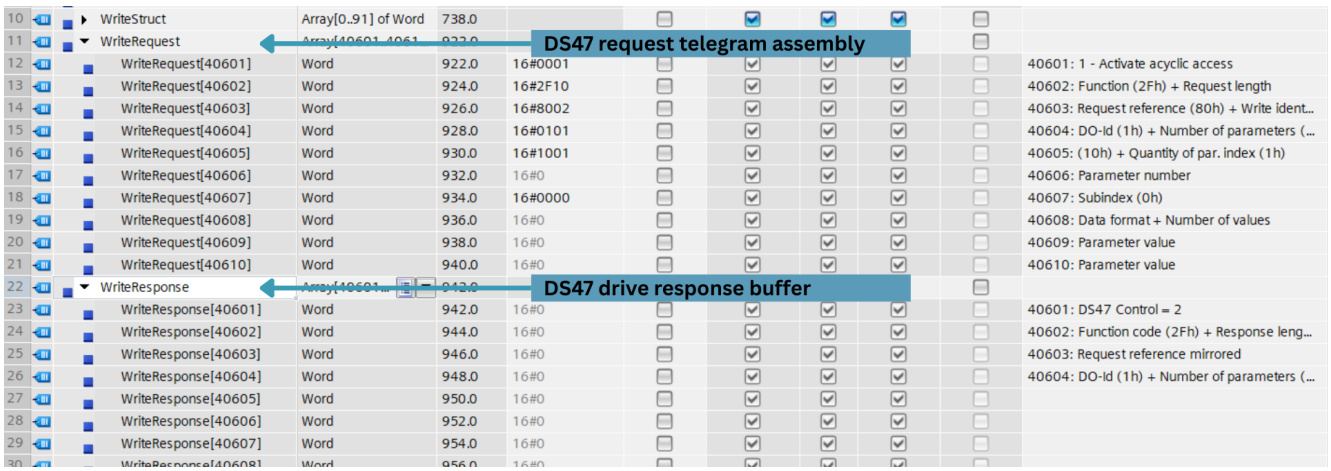
The communication architecture separates runtime parameter values, predefined parameters, and DS47 transaction data into dedicated write structures for deterministic parameter handling.



10	WriteStruct	Array[0..91] of Word	738.0		
11	WriteStruct[0]	Word	738.0	10#0003	40606: Parameter number
12	WriteStruct[1]	Word	740.0	16#0501	40608: Data format + Number of values
13	WriteStruct[2]	Word	742.0	16#0003	40609: Parameter value
14	WriteStruct[3]	Word	744.0	16#0000	40610: Parameter value
15	WriteStruct[4]	Word	746.0	10#0010	10#0010
16	WriteStruct[5]	Word	748.0	16#0501	10#0010
17	WriteStruct[6]	Word	750.0	16#0001	10#0010: 1 = Start comissioning
18	WriteStruct[7]	Word	752.0	16#0000	10#0010: 0 = fixed
19	WriteStruct[8]	Word	754.0	10#0100	10#0100
20	WriteStruct[9]	Word	756.0	16#0501	10#0100
21	WriteStruct[10]	Word	758.0	16#0	10#0100: Region
22	WriteStruct[11]	Word	760.0	16#0000	10#0100: 0 = fixed
23	WriteStruct[12]	Word	762.0	10#0301	10#0301
24	WriteStruct[13]	Word	764.0	16#0501	10#0301
25	WriteStruct[14]	Word	766.0	16#0000	10#0301: 0 = fixed
26	WriteStruct[15]	Word	768.0	16#0000	10#0301: 0 = fixed
27	WriteStruct[16]	Word	770.0	10#0304	10#0304
28	WriteStruct[17]	Word	772.0	16#0601	10#0304
29	WriteStruct[18]	Word	774.0	16#0	10#0304: Motor Voltage
30	WriteStruct[19]	Word	776.0	16#0000	10#0304: 0 = fixed
31	WriteStruct[20]	Word	778.0	10#0305	10#0305
32	WriteStruct[21]	Word	780.0	16#0801	10#0305
33	WriteStruct[22]	Word	782.0	16#0	10#0305: Motor Amp
34	WriteStruct[23]	Word	784.0	16#0	10#0305: Motor Amp

The DS47 communication architecture uses dedicated request and response transaction buffers for deterministic runtime parameter handling.

The PLC dynamically assembles DS47 telegrams, sends parameter transactions to the drive, and validates mirrored response data before completing the communication cycle.



10	WriteStruct	Array[0..91] of Word	738.0		
11	WriteRequest	Array[40601..4061...	922.0		
12	WriteRequest[40601]	Word	922.0	16#0001	40601: 1 - Activate acyclic access
13	WriteRequest[40602]	Word	924.0	16#2F10	40602: Function (2Fh) + Request length
14	WriteRequest[40603]	Word	926.0	16#8002	40603: Request reference (80h) + Write ident...
15	WriteRequest[40604]	Word	928.0	16#0101	40604: DO-Id (1h) + Number of parameters (...
16	WriteRequest[40605]	Word	930.0	16#1001	40605: (10h) + Quantity of par. index (1h)
17	WriteRequest[40606]	Word	932.0	16#0	40606: Parameter number
18	WriteRequest[40607]	Word	934.0	16#0000	40607: Subindex (0h)
19	WriteRequest[40608]	Word	936.0	16#0	40608: Data format + Number of values
20	WriteRequest[40609]	Word	938.0	16#0	40609: Parameter value
21	WriteRequest[40610]	Word	940.0	16#0	40610: Parameter value
22	WriteResponse	Array[40601...	942.0		
23	WriteResponse[40601]	Word	942.0	16#0	40601: DS47 Control = 2
24	WriteResponse[40602]	Word	944.0	16#0	40602: Function code (2Fh) + Response leng...
25	WriteResponse[40603]	Word	946.0	16#0	40603: Request reference mirrored
26	WriteResponse[40604]	Word	948.0	16#0	40604: DO-Id (1h) + Number of parameters (...
27	WriteResponse[40605]	Word	950.0	16#0	
28	WriteResponse[40606]	Word	952.0	16#0	
29	WriteResponse[40607]	Word	954.0	16#0	
30	WriteResponse[40608]	Word	956.0	16#0	

Dedicated request and response areas simplify transaction validation and runtime parameter processing.

5. DS47 Communication Workflow

The DS47 communication workflow is implemented as a state-machine sequence inside the PLC program.

Standard cyclic polling is executed continuously for normal drive control and monitoring. When a parameter read or write operation is requested from the HMI, the cyclic polling sequence reaches a controlled handover step where the acyclic DS47 transaction is executed.

During this phase, the PLC builds the DS47 request structure, writes it to the drive through Modbus, waits for the drive processing period, reads the response area, validates the response, and then returns to the normal cyclic polling sequence.

This approach prevents overlapping Modbus transactions and keeps both cyclic drive data and acyclic parameter access synchronized on the same communication channel.

Cyclic polling handover point for DS47 acyclic transaction

The cyclic polling sequence pauses at Step 8 to check whether runtime parameter access or parameter programming is required.

If an acyclic parameter request is pending, the PLC switches to the DS47 communication workflow. Otherwise, the scheduler continues polling the next drive in the sequence.

The acyclic transaction flags remain TRUE by default, allowing the cyclic polling scheduler to continue normal operation.

```
239
240     IF #MB_MASTER.ERROR THEN ... END_IF;
243
244 8:  // Acyclic wait
245
246     IF #Acyclic_RDY AND #Prog_RDY THEN
247         #SSW := 9;
248     END_IF;
249
250 9:  // Next drive
251
252     #DRIVE_INDEX := #DRIVE_INDEX + 1;
```

Check for pending DS47 transactions

Continue normal cyclic polling

When an HMI parameter access request is detected, the corresponding acyclic transaction flag is cleared and the PLC switches to the DS47 communication workflow for the selected drive.

```
313 REGION Programming
314
315     REGION COMMENTS
320
321     "SINAMICS_ACYCLIC".PROGRAMING[#DRIVE_INDEX] := "HMI".StartProgramming AND #DRIVE_INDEX = "HMI".ActDriveNum;
322     #Prog_RDY := NOT "SINAMICS_ACYCLIC".PROGRAMING[#DRIVE_INDEX] AND #SSW = 8;
323
324     FOR #i_cntr := 0 TO 10 DO
325         IF #i_cntr <> "HMI".ActDriveNum THEN
326             "SINAMICS_ACYCLIC".PROGRAMING[#i_cntr] := FALSE;
```

Runtime programming request generation

```
496 REGION Acyclic read
497
498     REGION COMMENTS
506
507     IF "SINAMICS_ACYCLIC".A... THEN ... END_IF;
517
518     IF "HMI".ActScreenNumb ... THEN ... END_IF;
521
522     IF "HMI".ActDriveNum <= 2 THEN ... END_IF;
527
528     IF NOT ... THEN ... END_IF;
533
534     IF "SINAMICS_ACYCLIC".READ[#DRIVE_INDEX] AND #SSW = 8 THEN
535
536         #Acyclic_RDY := FALSE;
537
538         // Main acyclic operation steps
539         CASE "SINAMICS_ACYCLIC".AState.SSW OF
540
```

Runtime parameter monitoring request

Start acyclic DS47 transaction

Acyclic communication is executed only for the active HMI-selected drive, preventing unnecessary Modbus transactions across the multi-drive network.

State-machine based DS47 transaction execution

The DS47 parameter programming workflow is implemented as a deterministic PLC state machine. Each transaction step is executed sequentially, including HMI parameter transfer, DS47 request assembly, Modbus transaction execution, response polling, and transaction completion.

This architecture prevents overlapping communication requests and maintains synchronized parameter handling across the multi-drive network.

```
363     // Main acyclic operation steps
364     CASE "SINAMICS_ACYCLIC".PState.SSW OF
365
366         0:  // Initial state
367             #Index := 0;
368
369         1: // Transfer new param to WriteStruct
370
371             "TransferParamInt"(ParPackSize := #PARAM_PACK_SIZE,
372                                WriteStructSize := #WRITE_STRUCT_SIZE,
373                                HmiParamInput_Pack := "HMI".ParametrsInput.Pack1);
374             "TransferParamInt"(ParPackSize := #PARAM_PACK_SIZE,
375                                WriteStructSize := #WRITE_STRUCT_SIZE,
376                                HmiParamInput_Pack := "HMI".ParametrsInput.Pack2);
377             "TransferParamReal"(ParPackSize := #PARAM_PACK_SIZE,
378                                 WriteStructSize := #WRITE_STRUCT_SIZE,
379                                 HmiParamInput_Pack := "HMI".ParametrsInput.Pack3);
380
381             "SINAMICS_ACYCLIC".PState.SSW := 2;
382
383         2:  // Write parameter request: Writing the parameter value
384
385             IF "HMI".ParametrsInput.Pack1[0].Value = 1 AND "SINAMICS_ACYCLIC".WriteStruct[#Index] = 308 THEN
386                 #Index += 4;
387             ELSIF "HMI".ParametrsInput.Pack1[0].Value <> 1 AND "SINAMICS_ACYCLIC".WriteStruct[#Index] = 309 THEN
388                 #Index += 4;
389             END_IF;
390
```

Sequential DS47 workflow execution

Runtime HMI parameter transfer

DS47 request telegram

Write transactions are processed one parameter at a time to simplify response validation and runtime error handling.

Automatic selection of parameter structure based on drive region configuration.

```
384
385         IF "HMI".ParametrsInput.Pack1[0].Value = 1 AND "SINAMICS_ACYCLIC".WriteStruct[#Index] = 308 THEN
386             #Index += 4;
387         ELSIF "HMI".ParametrsInput.Pack1[0].Value <> 1 AND "SINAMICS_ACYCLIC".WriteStruct[#Index] = 309 THEN
388             #Index += 4;
389         END_IF;
390
```

Parameter p0308 / p0309 interpretation changes depending on the selected regional standard:

- Power Factor representation
- Motor efficiency representation

Runtime scaling and parameter mapping are adjusted automatically before DS47 transaction execution.

Modbus transaction execution through MB_MASTER

The DS47 acyclic communication layer executes runtime parameter transactions through the Siemens MB_MASTER instruction.

Each transaction is processed sequentially, including telegram transfer, response monitoring, communication status verification, and DS47 response validation before continuing to the next parameter.

The workflow also includes communication error detection and parameter-level error interpretation to ensure reliable runtime drive programming and controlled execution of acyclic parameter operations.

```
434         4:  // Response for write
435
436             "MB_MASTER_DB"(REQ := #MB_MASTER.REQ,
437                            MB_ADDR := #DRIVE_ADDR,
438                            MODE := 0,
439                            DATA_ADDR := #Start,
440                            DATA_LEN := 14,
441                            DONE => #MB_MASTER.DONE,
442                            ERROR => #MB_MASTER.ERROR,
443                            STATUS => #MB_MASTER.STATUS,
444                            DATA_PTR := "SINAMICS_ACYCLIC".WriteResponse);
445
446             #MB_MASTER.REQ := TRUE;
447
448             IF #MB_MASTER.DONE THEN
449                 "SINAMICS_ACYCLIC".PState.OkR += 1;
450             END_IF;
451
452             IF #MB_MASTER.ERROR THEN
453                 "SINAMICS_ACYCLIC".PState.WrnR += 1;
454             END_IF;
455
456             IF "SINAMICS_ACYCLIC".WriteResponse[40601] = 16#0002 AND "SINAMICS_ACYCLIC".WriteResponse[40603] = 16#8002 TH
457                 IF #Index >= (#WRITE_STRUCT_SIZE - 3) THEN
458                     "SINAMICS_ACYCLIC".PState.SSW := 0;
459                     "HMI".StartProgramming := FALSE;
460                     #Prog_RDY := TRUE;
461                 ELSE
462                     #Index += 4;
463                     "SINAMICS_ACYCLIC".PState.SSW := 2;
464                 END_IF;
465             ELSIF "SINAMICS_ACYCLIC".WriteResponse[40601] = 16#0002 AND "SINAMICS_ACYCLIC".WriteResponse[40603] = 16#8082
466                 #WrittenWithErr := TRUE;
467                 #WrittenWithErrPar := WORD_TO_INT(IN := "SINAMICS_ACYCLIC".WriteRequest[40606]);
468                 #WrittenWithErrorDesc := WORD_TO_INT(IN := "SINAMICS_ACYCLIC".WriteResponse[40607]);
469                 IF #ACK_WRITE_ERR THEN
```

6. Parameter Structures

The DS47 communication framework is organized as a structured register-based telegram architecture separating request generation, response handling, runtime parameter storage, and engineering value conversion. Independent transaction areas are maintained for read and write operations to simplify sequential processing and diagnostics during runtime parameter access.

The implementation currently supports the DS47 parameter formats 0501, 0601, and 0801 identified during reverse engineering and runtime testing of the SINAMICS V20 communication layer. These formats are used for single-byte parameters, full-word integer parameters, and IEEE754 floating-point values respectively.

```
34    "SINAMICS_V20_DB"(HW_ID := "Local~CB_1241_(RS485)",
35                      NUMBER_OF_DRIVES := 7,
36                      START_DRIVE_ADDR := 10,
37                      WRITE_STRUCT_SIZE := 91,
38                      PARAM_PACK_SIZE := 16,
39                      COMISS_SCREEN_NUM := 4,
40                      PROG_SCREEN_NUM := 5,
41                      FAULTS_RESET_VFD := "HMI".DriveFltReset);
42
```

WriteStruct Array Max index

ReadResult Array Max index

Read Transaction Structure

Read transactions support multi-parameter telegram packing to reduce communication overhead and improve runtime efficiency. Write transactions are executed sequentially with one parameter per transaction to simplify response validation and parameter-level error handling.

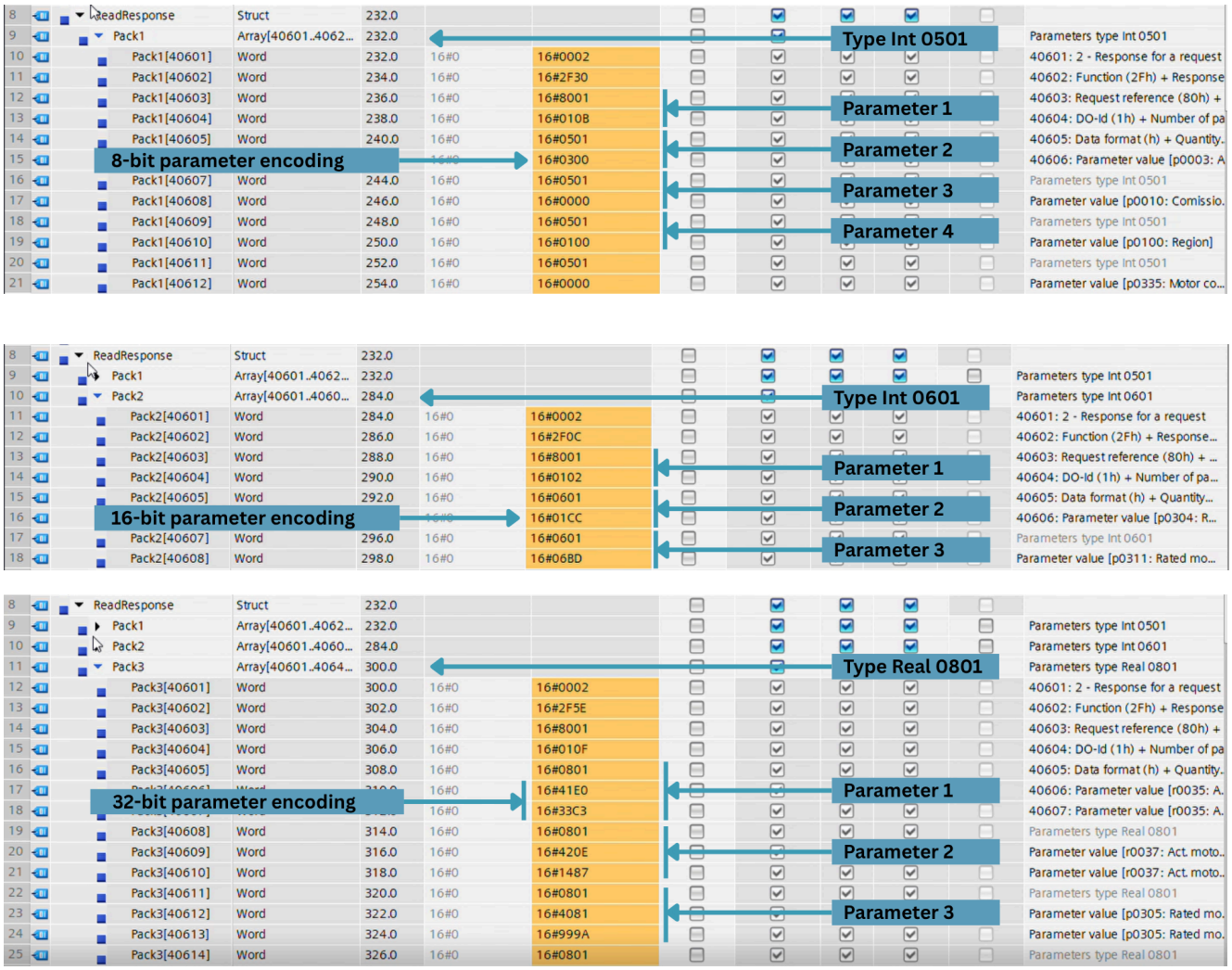
7	ReadRequest	Struct	40.0			
8	Pack1	Array[40601..4063...	40.0			Type Int
9	Pack1[40601]	Word	40.0	16#0001	16#0001	40601: 1 - Activate acyclic access
10	Pack1[40602]	Word	42.0	16#0	16#2F46	40602: Function (2Fh) + Request I.
11	Pack1[40603]	Word	44.0	16#8001	16#8001	40603: Request reference (80h) +
12	Pack1[40604]	Word	46.0	16#0	16#010B	Message header
13	Pack1[40605]	Word	48.0	16#1001	16#1001	40605: (10h) + Quantity of par. in...
14	Pack1[40606]	Word	50.0	10#0003	16#0003	Parameter 1
15	Pack1[40607]	Word	52.0	16#0000	16#0000	40607: Subindex (0h)
16	Pack1[40608]	Word	54.0	16#1001	16#1001	Parameters type Int 0501
17	Pack1[40609]	Word	56.0	10#0010	16#000A	Parameter 2
18	Pack1[40610]	Word	58.0	16#0000	16#0000	Parameters type Int 0501
19	Pack1[40611]	Word	60.0	16#1001	16#1001	Parameters type Int 0501
20	Pack1[40612]	Word	62.0	10#0100	16#0064	Parameter 3
21	Pack1[40613]	Word	64.0	16#0000	16#0000	Parameters type Int 0501

7	ReadRequest	Struct	40.0			
8	Pack1	Array[40601..4063...	40.0			Parameters type Int 0501
9	Pack2	Array[40601..4061...	114.0			Type Real
10	Pack3	Array[40601..4064...	134.0			Parameters type Real 0801
11	Pack3[40601]	Word	134.0	16#0001	16#0001	40601: 1 - Activate acyclic access
12	Pack3[40602]	Word	136.0	16#0	16#2F5E	40602: Function (2Fh) + Request I.
13	Pack3[40603]	Word	138.0	16#8001	16#8001	40603: Request reference (80h) +
14	Pack3[40604]	Word	140.0	16#0	16#010F	Message header
15	Pack3[40605]	Word	142.0	16#1001	16#1001	40605: (10h) + Quantity of par. in...
16	Pack3[40606]	Word	144.0	10#0035	16#0023	Parameter 1
17	Pack3[40607]	Word	146.0	16#0000	16#0000	40607: Subindex (0h)
18	Pack3[40608]	Word	148.0	16#1001	16#1001	Parameters type Real 0801
19	Pack3[40609]	Word	150.0	10#0037	16#0025	Parameter 2
20	Pack3[40610]	Word	152.0	16#0000	16#0000	Parameters type Real 0801
21	Pack3[40611]	Word	154.0	16#1001	16#1001	Parameters type Real 0801
22	Pack3[40612]	Word	156.0	10#0305	16#0131	Parameter 3
23	Pack3[40613]	Word	158.0	16#0000	16#0000	Parameters type Real 0801

Implemented DS47 formats:

- 0501 → 8-bit parameter encoding
- 0601 → 16-bit parameter encoding
- 0801 → 32-bit floating-point encoding

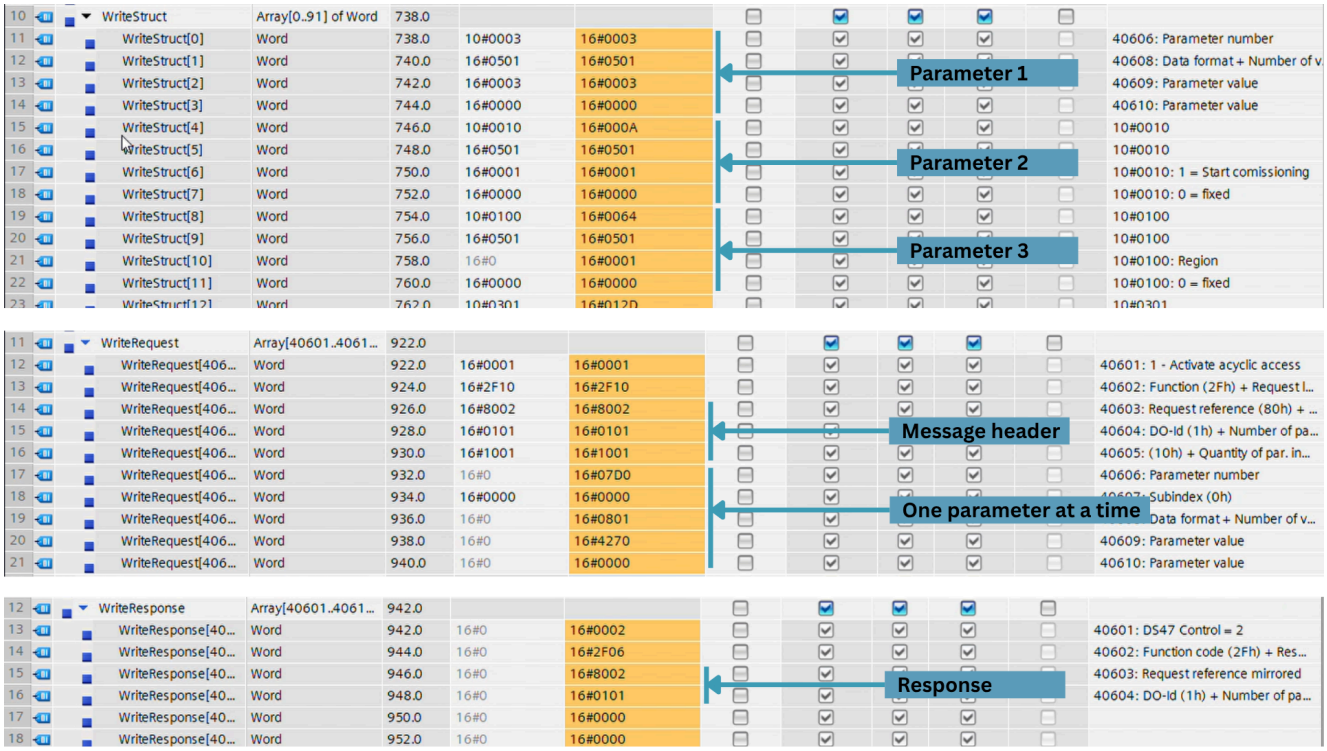
Example of DS47 read response telegram structures:



The framework automatically converts raw DS47 telegram values into engineering runtime representations accessible from the HMI and PLC logic layer.

Write Transaction Structure

Read operations support multi-parameter telegram packing for optimized runtime communication. In contrast, write operations are executed sequentially with one parameter per transaction to simplify response validation, parameter confirmation, and runtime error interpretation.


Parameter 1
Parameter 2
Parameter 3
Message header
One parameter at a time
Response

7. Multi-Drive Scheduling

The framework supports scalable multi-drive communication through configurable drive indexing and dynamic Modbus address generation. Cyclic polling and acyclic DS47 communication are coordinated through a centralized runtime scheduler operating over a shared RS485 communication channel.

Each drive is assigned a sequential communication index and dedicated runtime data storage area, simplifying diagnostics, parameter monitoring, and runtime visualization.

```
33
34      "SINAMICS_V20_DB"(HW_ID := "Local~CB_1241_(RS485)",
35                        NUMBER_OF_DRIVES := 7,
36                        START_DRIVE_ADDR := 10,
37                        WRITE_STRUCT_SIZE := 91,
38                        PARAM_PACK_SIZE := 16,
39                        COMISS_SCREEN_NUM := 4,
40                        PROG_SCREEN_NUM := 5,
41                        FAULTS_RESET_VFD := "HMI".DriveFltReset);
```

Configurable drive network size

Modbus drive addressing start

The cyclic communication layer continuously rotates through all configured drives using sequential drive indexing. Each communication cycle dynamically updates the Modbus target address and executes polling operations individually for every drive.

The scheduler supports automatic drive exclusion, polling restart handling, and runtime communication arbitration while maintaining deterministic communication timing across the RS485 network.

```
244         8:  // Acyclic wait
245
246             IF #Acyclic_RDY AND #Prog_RDY THEN
247                 #SSW := 9;
248             END_IF;
249
250         9:  // Next drive
251
252             #DRIVE_INDEX := #DRIVE_INDEX + 1;
253
254             IF #DRIVE_INDEX >= #NUMBER_OF_DRIVES THEN
255                 #DRIVE_INDEX := 0;
256             END_IF;
257
258             // Always skip index 3
259             IF #DRIVE_INDEX = 3 THEN
260                 #DRIVE_INDEX := #DRIVE_INDEX + 1;
261                 CONTINUE;
262             END_IF;
263
264             #SSW := 0;
265
266     END_CASE;
```

Acyclic communication

Sequential drive polling rotation

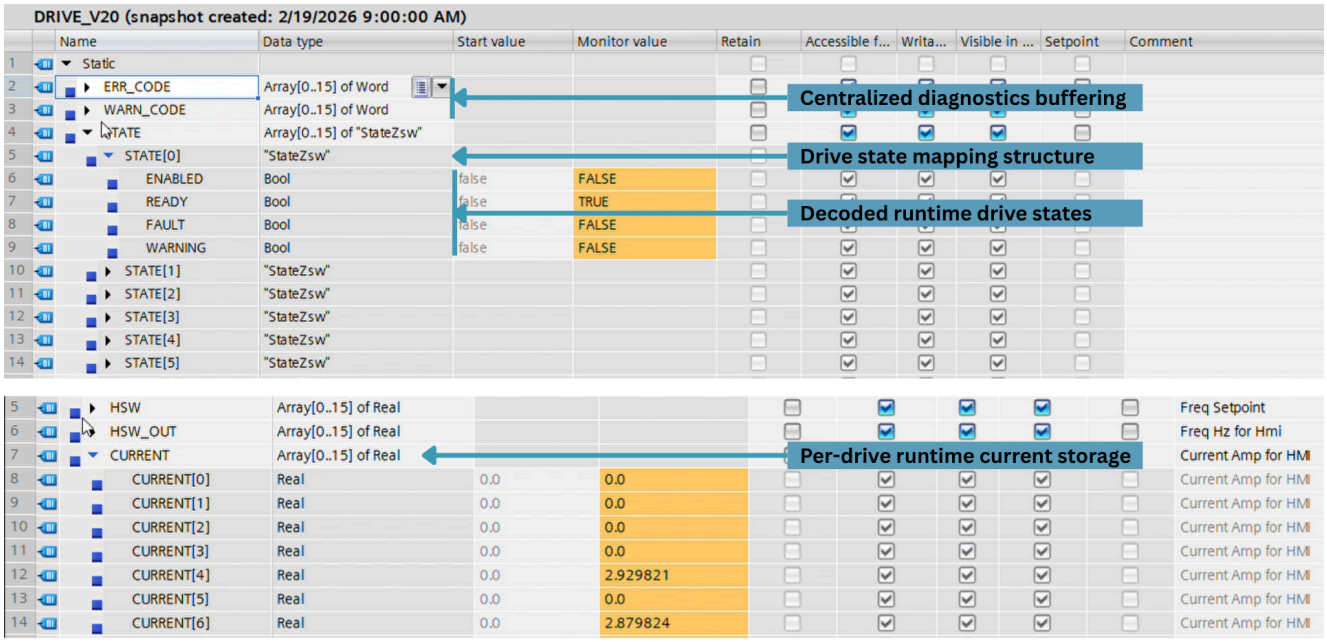
Automatic polling loop restart

Reserved / disabled drive exclusion

Return to start cyclic communication state

Unlike the acyclic DS47 parameter layer, which temporarily reserves communication access for a single drive, the cyclic communication layer continuously polls all configured drives and stores runtime values into centralized PLC data structures.

The framework maintains dedicated runtime arrays for status words, warnings, errors, frequency references, and engineering values, enabling simultaneous HMI visualization and PLC logic access across the complete drive network.



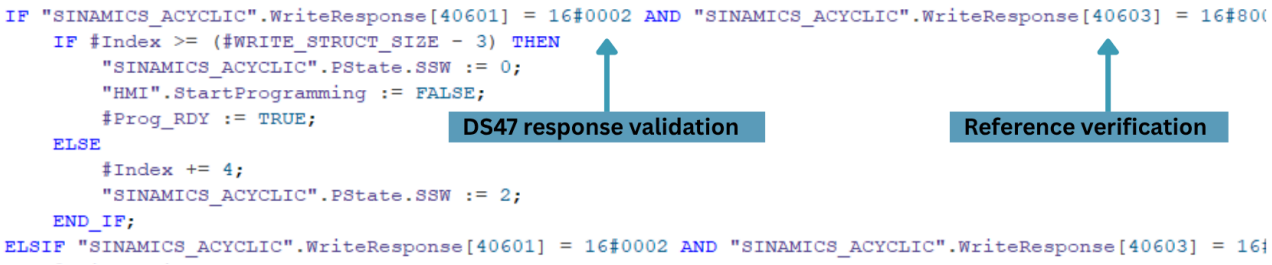
Acyclic parameter communication operates independently from the cyclic polling layer and temporarily reserves the communication channel for a single drive transaction.

8. Diagnostics & Error Handling

DS47 Response Validation

Each DS47 parameter transaction is validated through response telegram verification before the next operation is executed. The framework checks both DS47 control responses and mirrored request references to ensure transaction integrity and deterministic parameter programming behavior.

Only validated responses are allowed to continue the sequential parameter execution workflow.

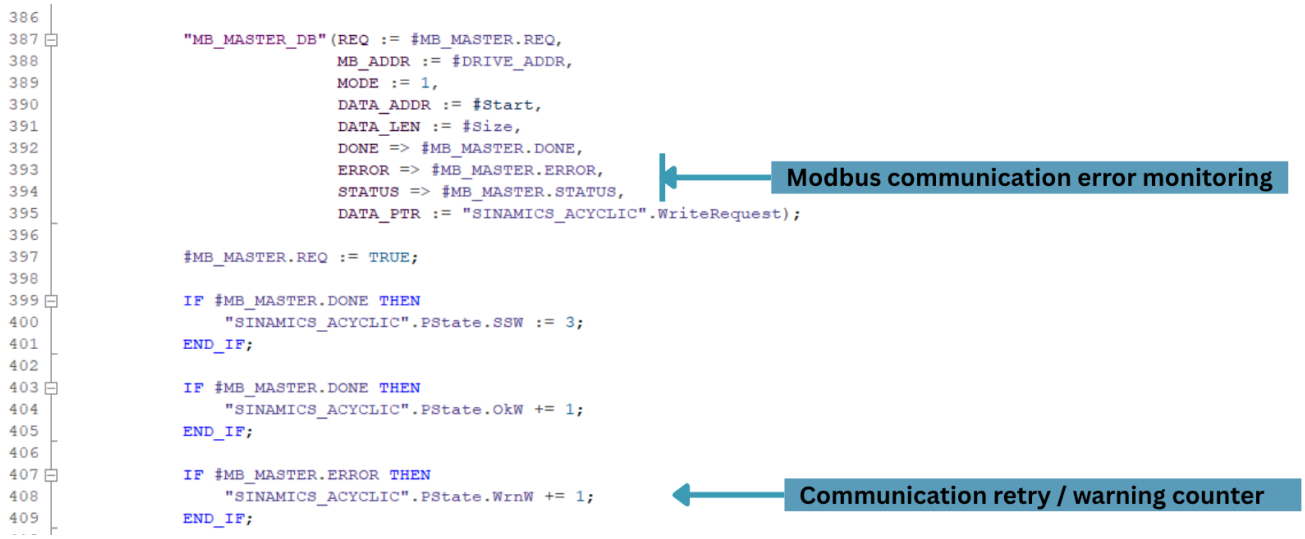



```
IF "SINAMICS_ACYCLIC".WriteResponse[40601] = 16#0002 AND "SINAMICS_ACYCLIC".WriteResponse[40603] = 16#80(
    IF #Index >= (#WRITE_STRUCT_SIZE - 3) THEN
        "SINAMICS_ACYCLIC".PState.SSW := 0;
        "HMI".StartProgramming := FALSE;
        #Prog_RDY := TRUE;
    ELSE
        #Index += 4;
        "SINAMICS_ACYCLIC".PState.SSW := 2;
    END_IF;
ELSIF "SINAMICS_ACYCLIC".WriteResponse[40601] = 16#0002 AND "SINAMICS_ACYCLIC".WriteResponse[40603] = 16#
```

Communication Error Detection

The communication layer continuously monitors MB_MASTER execution status and detects Modbus runtime communication failures during cyclic and acyclic transactions.

Communication faults are captured through dedicated diagnostic counters and runtime status handling, allowing controlled recovery without interrupting the complete polling scheduler.

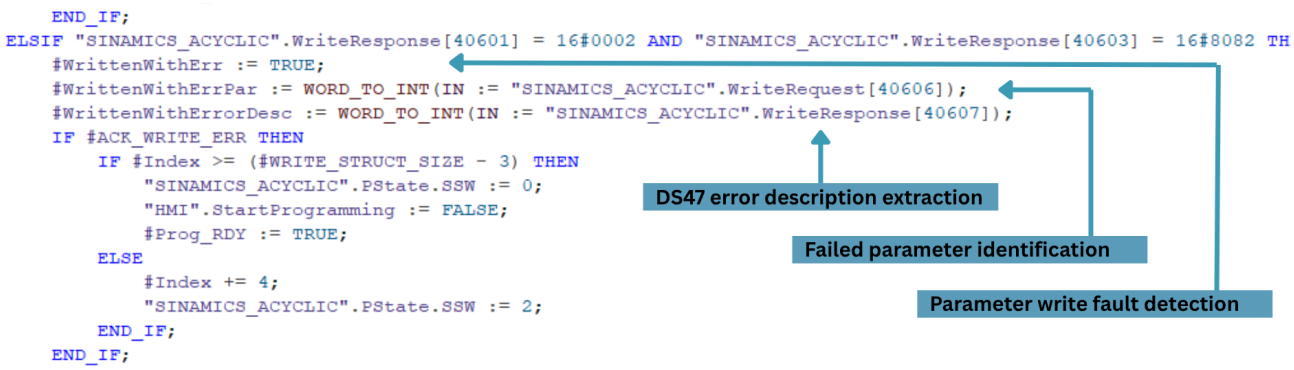



```
386
387         "MB_MASTER_DB"(REQ := #MB_MASTER.REQ,
388                        MB_ADDR := #DRIVE_ADDR,
389                        MODE := 1,
390                        DATA_ADDR := #Start,
391                        DATA_LEN := #Size,
392                        DONE => #MB_MASTER.DONE,
393                        ERROR => #MB_MASTER.ERROR,
394                        STATUS => #MB_MASTER.STATUS,
395                        DATA_PTR := "SINAMICS_ACYCLIC".WriteRequest);
396
397         #MB_MASTER.REQ := TRUE;
398
399         IF #MB_MASTER.DONE THEN
400             "SINAMICS_ACYCLIC".PState.SSW := 3;
401         END_IF;
402
403         IF #MB_MASTER.DONE THEN
404             "SINAMICS_ACYCLIC".PState.OkW += 1;
405         END_IF;
406
407         IF #MB_MASTER.ERROR THEN
408             "SINAMICS_ACYCLIC".PState.WrnW += 1;
409         END_IF;
```

Parameter-Level Error Interpretation

The framework supports parameter-level diagnostic interpretation by extracting DS47 error information directly from the response telegram structure.

Failed write operations are linked to the associated parameter number and DS47 error code, simplifying runtime troubleshooting and commissioning diagnostics from the HMI layer.

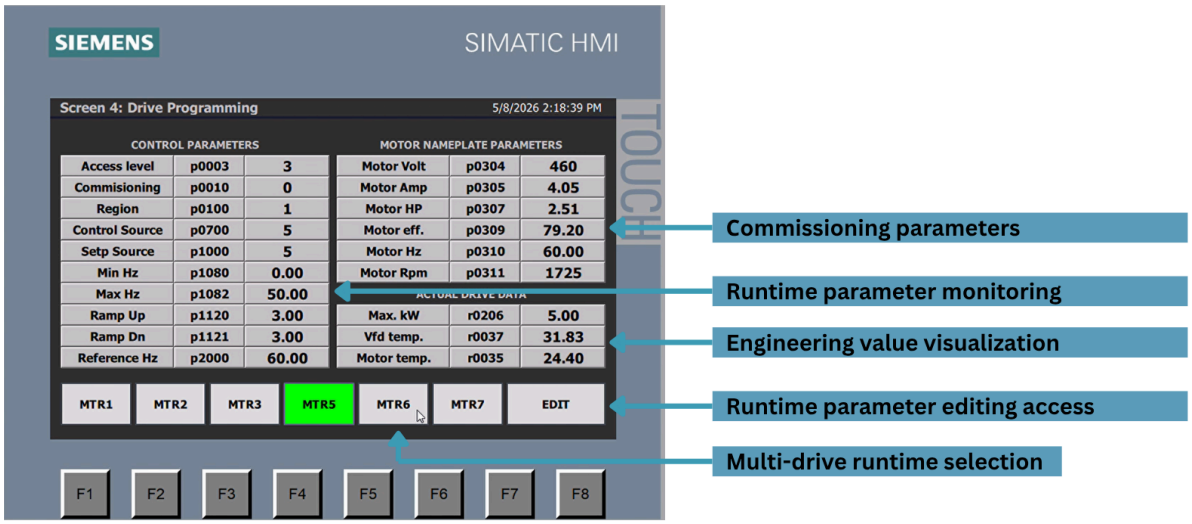


The diagnostic framework was designed to support runtime commissioning, parameter editing, and fault isolation without requiring direct drive access through local keypad operation.

9. Runtime HMI

Runtime Monitoring Interface

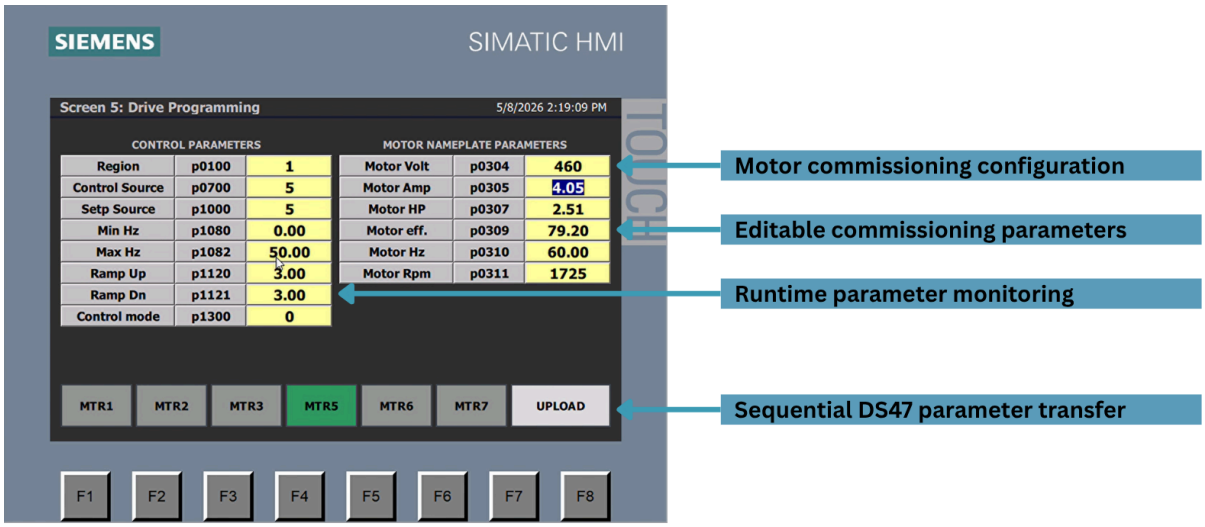
The runtime HMI interface provides centralized monitoring and parameter access for all connected SINAMICS V20 drives. Engineering values, commissioning parameters, and runtime diagnostics are continuously updated through the cyclic and acyclic communication layers.



The interface allows operators and commissioning personnel to navigate between drives and access runtime parameter data without direct drive interaction.

Runtime Parameter Editing

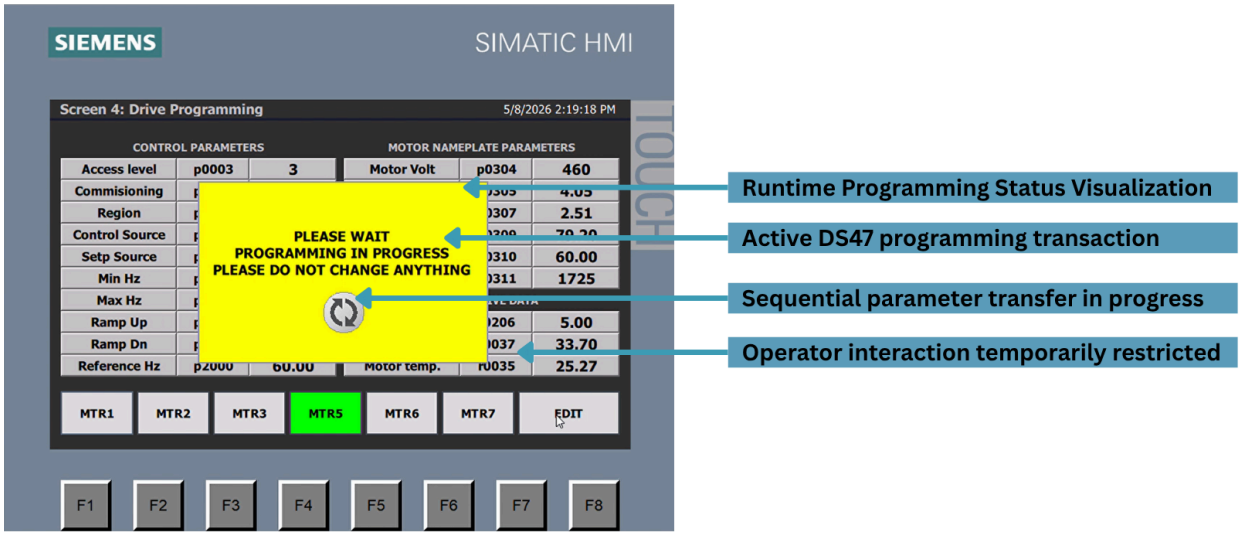
The HMI supports runtime parameter modification through the DS47 acyclic communication framework. Parameter changes are validated and transferred sequentially to the selected drive while maintaining communication integrity and runtime diagnostics visibility.



The framework also supports region-dependent parameter handling and automatic engineering value conversion for different parameter formats.

Runtime Programming Status Visualization

The HMI runtime layer provides visual feedback during active DS47 parameter programming operations. Sequential parameter transfers are executed while continuously monitoring communication responses and transaction completion status.



Operator interaction is temporarily restricted during programming execution to maintain deterministic communication behavior and prevent interruption of the active transaction sequence.

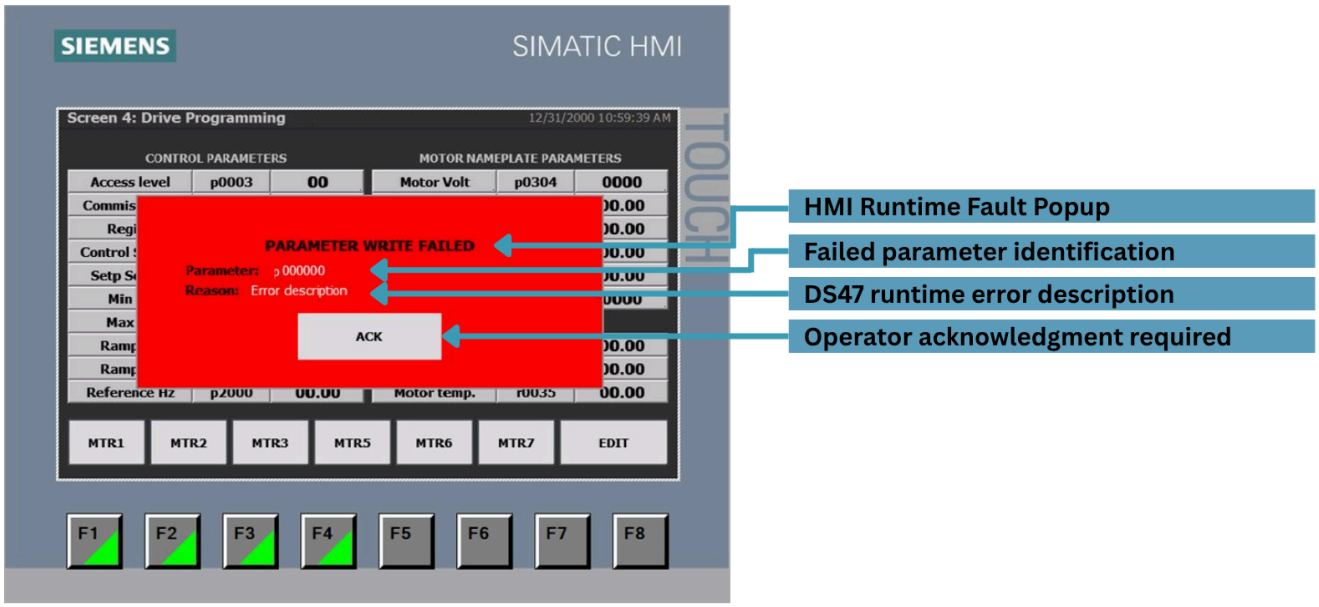
During runtime programming operations, the HMI automatically returns to the active programming screen to ensure visibility of the current transaction state and prevent unintended operator actions.

The runtime HMI workflow was designed to support safe commissioning operations while maintaining visibility, diagnostic transparency, and controlled operator interaction during active parameter programming.

Runtime Error Handling

Runtime parameter programming faults are displayed directly through the HMI diagnostic layer. Failed parameter operations include both the associated parameter number and DS47 runtime error description to simplify commissioning diagnostics and troubleshooting.

The workflow requires operator acknowledgment before allowing further parameter transactions, preventing uncontrolled runtime programming continuation after communication or parameter-level failures.



The SINAMICS V20 documentation does not explicitly define DS47 parameter error codes for Modbus RTU acyclic communication.

Error interpretation within this framework was validated using the DS47 communication model documented for SINAMICS V90 drives, where the protocol structure and response behavior are equivalent. Runtime testing confirmed consistent error behavior for parameter validation and communication fault scenarios.

10. Document Revision History

Version	Date	Modifications
Rev.0	05//2026	Initial release